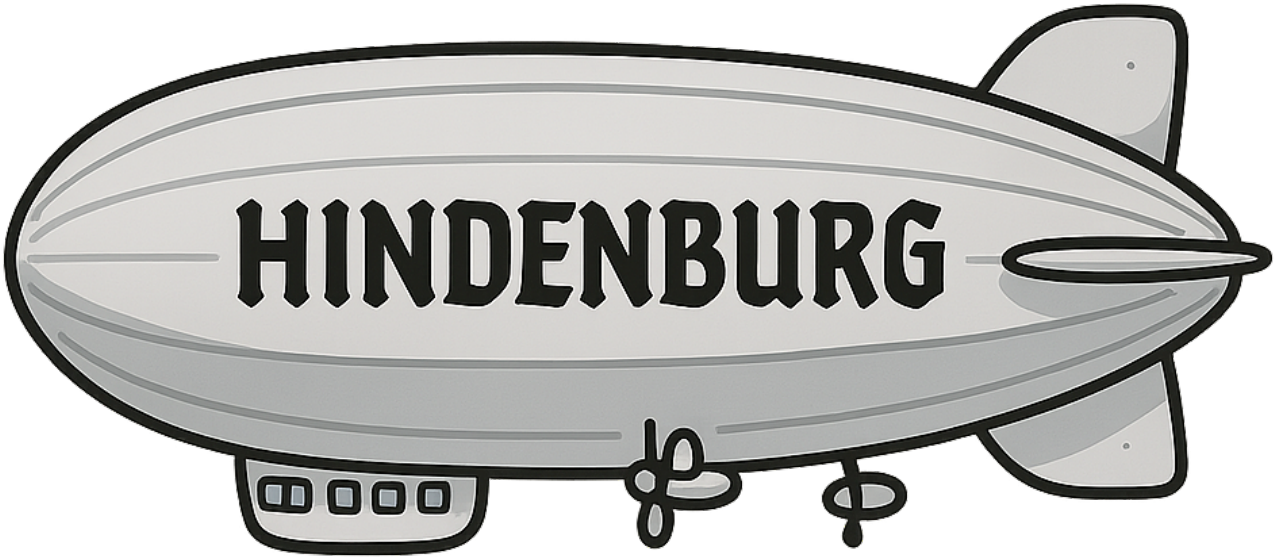
Autonome Hindenburg

Minor Autonome Systemen: Robotic Project Rapport



Bastiaan Buurman

Bart-Jan van Strien

Patrick de Kok & Julian Muller
Hogeschool van Amsterdam
Minor Autonome Systemen

Abstract

Dit rapport beschrijft het projectverloop van het vak Robotic Project binnen de minor Autonome Systemen.

16 juni 2025

Inhoudsopgave

1 Probleemstelling **3**

2 Methodologie: Vergelijking en keuze van positiebepaling **4**
2.1 GPS . . . 4
2.2 Radar . . . 4
2.3 Optisch . . . 4

3 Implementatie van de optische locatiebepaling **5**
3.1 Beelden stitchen . . . 5
3.2 Beelden verkrijgen . . . 7
3.3 Locatiebepaling . . . 8
3.4 Optische lokalisatie in een GUI . . . 9
3.4.1 Padfiltering: moving–average en spike–verwijdering . . . 10
3.4.2 GUI–ontwerp met PyQt5 . . . 10

4 Resultaatanalyse **11**
4.1 Kwalitatieve analyse . . . 11
4.2 Kwantitatieve analyse . . . 13
4.2.1 Frequentie en omvang van frameskips . . . 13
4.2.2 Effect op padnauwkeurigheid . . . 13

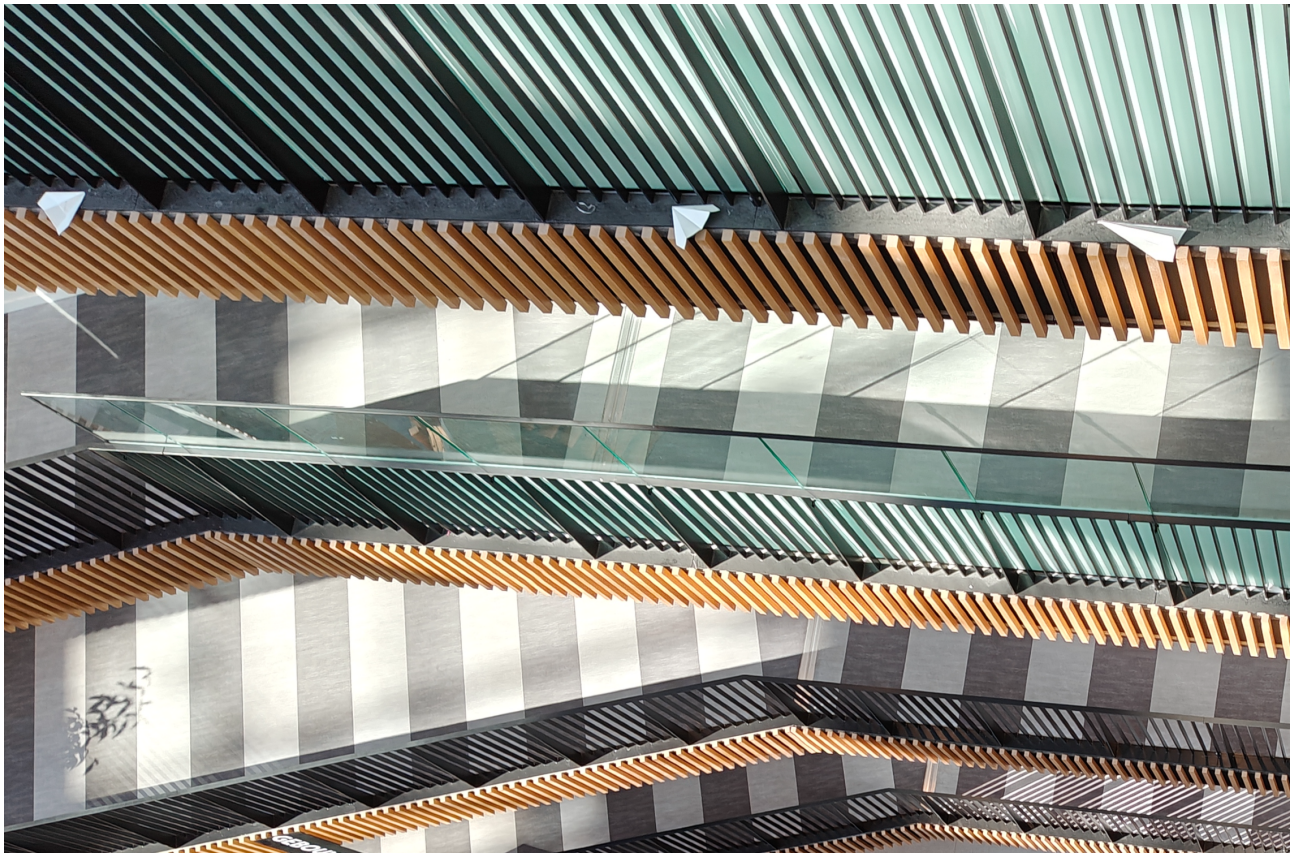
5 Conclusie **14**

Bronnen **15**

1. Probleemstelling

Binnen het atrium van het Jakoba Mulderhuis (JMH) moet een zeppelin autonoom kunnen navigeren zonder te botsen met omliggende structuren zoals de gevel van het Theo Thijssenhuis, de glazen achterzijde van het JMH en de open werkplekken. Het huidige probleem is dat er geen betrouwbare methode beschikbaar is om de positie en oriëntatie van de zeppelin nauwkeurig te bepalen binnen deze complexe binnenshuissituatie. Dit vormt een belemmering voor veilige en efficiënte autonome navigatie, omdat conventionele technieken zoals GPS en radar binnenshuis onvoldoende nauwkeurig of toepasbaar zijn. Het ontbreken van een effectieve locatiebepalingstechniek beperkt daarmee de operationele mogelijkheden van de zeppelin in het atrium en verhoogt het risico op botsingen en schade.

Deze probleemstelling is relevant omdat het veilig en autonoom laten navigeren van de zeppelin binnen het atrium een essentiële eerste stap is voor het zelfstandig opruimen van papieren vliegtuigen op de gevels (zie figuur 1).



Figuur 1: Papieren vliegtuigen op de gevels

2. Methodologie: Vergelijking en keuze van positiebepaling

Om de autonome vlucht van de zeppelin binnen het atrium van het JMH te realiseren, is gekozen voor een methodologische aanpak die zich richt op de vergelijking en selectie van geschikte positiebepalingsmethoden. Hierbij is het PEAS-model (Performance, Environment, Actuators, Sensors) als raamwerk gebruikt om het systeem te beschrijven en de relevante keuzes te structureren. De methodologie richt zich specifiek op de sensorkeuze, omdat de omgeving en actuatoren grotendeels door de opdrachtgever zijn vastgesteld.

2.1 GPS

In eerste instantie is door de opdrachtgever gekeken naar Global Positioning System (GPS) als locatiebepalingstechniek. GPS is een wereldwijd navigatiesysteem dat satellietsignalen gebruikt om de positie van een ontvanger te bepalen. Binnen een gebouw is het echter vrijwel onmogelijk om een nauwkeurige GPS-ontvangst te krijgen vanwege de signaalverzwakking en reflecties veroorzaakt door muren en andere constructiematerialen [1]. Hierdoor ontstaan onnauwkeurigheden die de betrouwbaarheid van de locatiebepaling sterk verminderen. Bovendien is de nauwkeurigheid van standaard GPS-signalen meestal beperkt tot enkele meters, wat onvoldoende is voor veilige autonome navigatie in een beperkte, complexe binnenruimte zoals het atrium.

2.2 Radar

Een alternatieve optie is radar, waarbij radargolven worden uitgezonden en de terugkaatsing wordt gemeten om afstanden tot objecten te bepalen. Theoretisch kan radar gebruikt worden om de afstand tot muren en andere structuren te bepalen, wat kan bijdragen aan positiebepaling binnen het gebouw. In de praktijk is deze methode echter sterk afhankelijk van de eigenschappen van de oppervlakken waartegen de golven weerkaatsen. Variaties in materiaal, textuur en hoek van de wanden kunnen de radarsignalen verstrooien of verzwakken, waardoor de gemeten afstanden onnauwkeurig worden. Daarnaast is radar vaak minder geschikt om complexe binnenomgevingen met veel reflecties en transparante oppervlakken, zoals glas, betrouwbaar in kaart te brengen [3].

2.3 Optisch

Een veelbelovende methode is optische locatiebepaling, waarbij camera's en beeldherkenningstechnieken worden gebruikt om de positie te bepalen door het vergelijken van actuele beelden met een vooraf gemaakte kaart van de omgeving. Digital Scene Matching Area Correlation (DSMAC) is een specifieke techniek die beeldcorrelatie toepast om de locatie te bepalen aan de hand van visuele kenmerken in het terrein of gebouw [2]. DSMAC werd oorspronkelijk ontwikkeld voor militaire toepassingen, zoals precisie-navigatie van kruisraketten, en is effectief gebleken ondanks veranderingen in lichtcondities en perspectief. Door een gedetailleerde visuele kaart van het atrium te maken, kan de zeppelin zijn positie nauwkeurig bepalen door continue beeldvergelijking.

3. Implementatie van de optische locatiebepaling

De grootste uitdaging bij de implementatie van de locatiebepaling binnen het JMH was het verzamelen van goede data. Vergelijkbare implementaties vinden doorgaans buiten plaats, waarbij satellietfoto's worden gebruikt als kaart van het terrein. Deze satellietfoto's zijn nauwkeurig gelabeld met bijbehorende coördinaten. Omdat er geen kaart met het bovenaanzicht van het JMH beschikbaar is, zal het eerste en grootste deel van de implementatie gericht zijn op het verkrijgen van een goede dataset.

3.1 Beelden stitchen

Het atrium in het JMH is groot en heeft een dak, waardoor het onmogelijk is om een enkele foto als kaart te gebruiken. In plaats daarvan wordt een opname gemaakt in banen in de lengte. Uit deze opnames worden de individuele frames in de volgorde van de video aan elkaar geplakt. Doordat de frames uit een video komen, staan ze op volgorde, wat het aantal mogelijkheden om foto's aan elkaar te plakken verkleint. Het aan elkaar plakken van foto's wordt *stitchen* genoemd.

Er worden steeds twee beelden aan elkaar geplakt, vergelijkbaar met het maken van een panorama op een telefoon. Eerst worden met Schaal-invariante Functietransformatie (SIFT) herkenningspunten uit de afbeeldingen gehaald (zie listing 1). Het selecteren van deze herkenningspunten gebeurt op basis van contrastverschillen, die onafhankelijk van schaalgrootte en rotatie te herkennen zijn.

```
def get_keypoint(left_img, right_img):
    l_img = cv2.cvtColor(left_img, cv2.COLOR_BGR2GRAY)
    r_img = cv2.cvtColor(right_img, cv2.COLOR_BGR2GRAY)

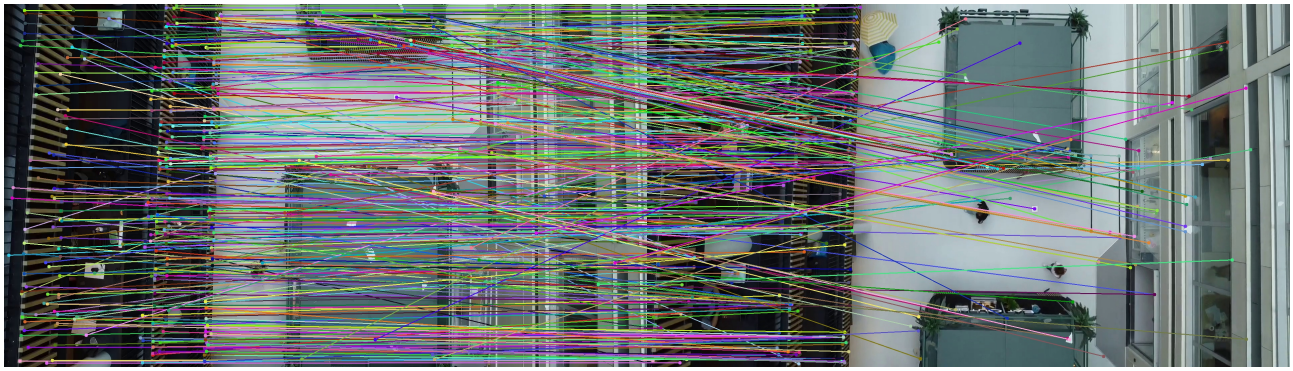
    sift = cv2.SIFT_create()
    key_points1, descriptor1 = sift.detectAndCompute(l_img, None)
    key_points2, descriptor2 = sift.detectAndCompute(r_img, None)
    return key_points1, descriptor1, key_points2, descriptor2
```

Listing 1: Verkrijgen van knooppunten uit de twee beelden met *get_keypoint* als onderdeel van het afbeelding stitchen

Van alle matches bepaalt de functie *match_keypoint* (zie listing 2) de goede matches door te controleren of de beste overeenkomst duidelijk beter is dan de tweede beste, gemeten met de ratio van hun descriptor-afstanden. Als de afstand van de beste match minder is dan 0,75 keer die van de op één na beste, wordt de match als betrouwbaar beschouwd. Dit voorkomt dat vage of onduidelijke matches (zoals personen) worden meegenomen en verhoogt zo de nauwkeurigheid van het samenvoegen van beelden. Zie figuur 2 voor een concreet voorbeeld.

```
def match_keypoint(key_points1, key_points2, descriptor1, descriptor2):
    good_matches = []
    for m, n in all_matches:
        if m[2] < 0.75 * n[2]:
            left_pt = key_points1[m[0]].pt
            right_pt = key_points2[m[1]].pt
            good_matches.append([left_pt[0], left_pt[1], right_pt[0], right_pt[1]])
    return good_matches
```

Listing 2: Verkrijgen van goede knooppunten als onderdeel van de functie *match_keypoint*



Figuur 2: Goede overeenkomende matches in de afbeelding.

De goede matchpunten worden gebruikt om een homografiematrix te berekenen. Homografie is een projectieve transformatie tussen twee beelden. Het linkerbeeld wordt getransformeerd zodat het precies over het rechterbeeld past. Hierdoor kunnen de beelden geometrisch op elkaar worden uitgelijnd. Vervolgens wordt het linkerbeeld met deze transformatie over het rechterbeeld gelegd, waarbij de overlappingen worden aangepast zodat de overgang tussen de beelden vloeiend en naadloos oogt.

Om de homografie robuust te schatten, wordt het RANSAC-algoritme toegepast. RANSAC (Random Sample Consensus) is een iteratief algoritme dat probeert de beste homografiematrix te vinden ondanks de aanwezigheid van foutieve matches (outliers). Het algoritme selecteert herhaaldelijk willekeurig een kleine subset van vier matchpunten om een voorlopige homografie te berekenen. Vervolgens wordt geteld hoeveel van de overige matchpunten binnen een bepaalde afstandsdrempel (threshold) passen bij deze homografie; deze punten worden *inliers* genoemd. De homografie met het grootste aantal inliers wordt gekozen als de beste schatting, omdat deze het meest consistent is met de meerderheid van de data. Hierdoor minimaliseert RANSAC de invloed van verkeerde matches en verhoogt het de nauwkeurigheid van het uitlijnen van de beelden. Zie listing 3 voor de implementatie en de kwalitatieve analyse voor het eindresultaat.

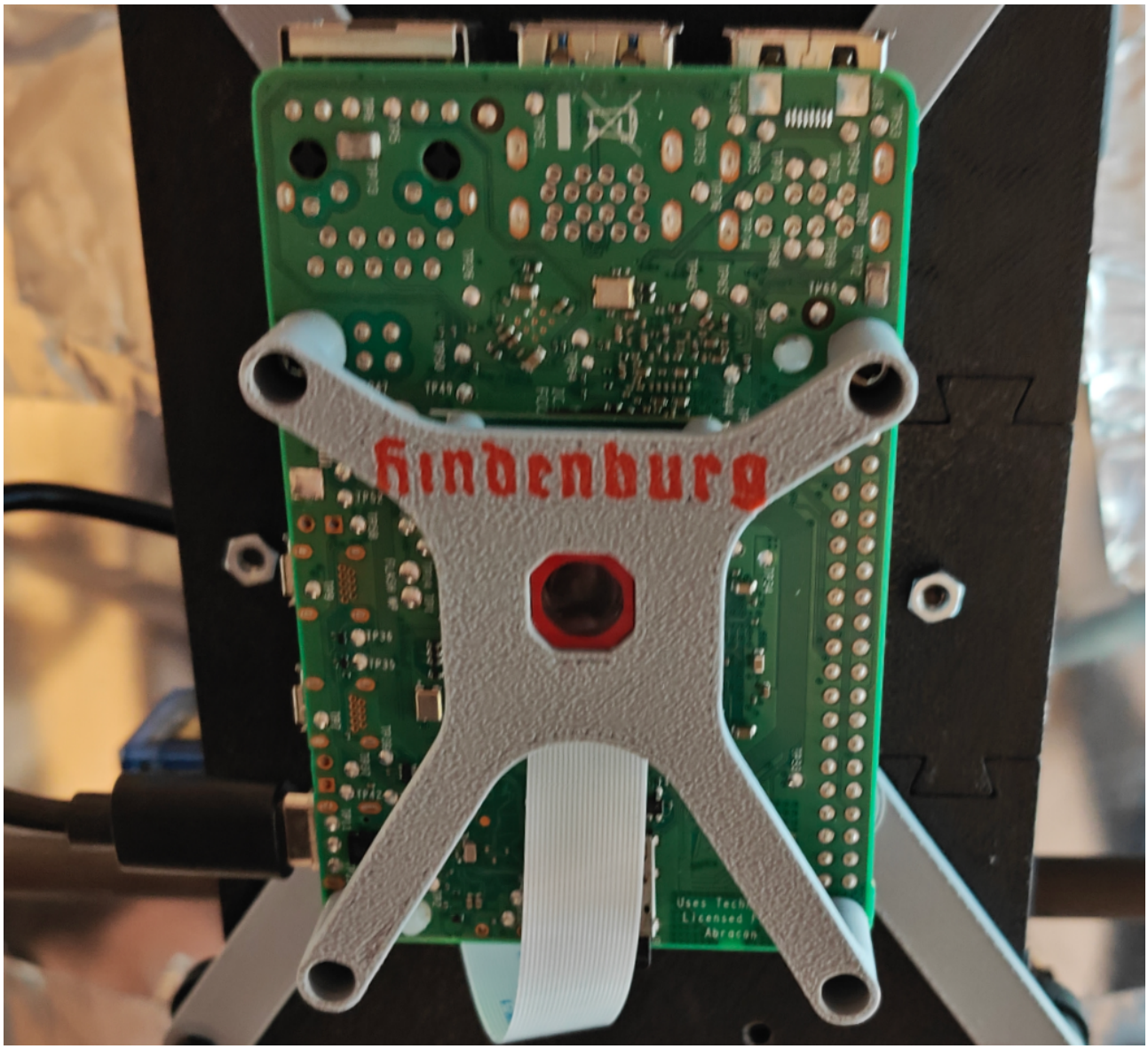
```
def ransac(good_pts, iterations=5000, threshold=5):
    best_inliers = []
    final_H = None
    for _ in tqdm(range(iterations), desc="RANSAC iteraties", unit="iter"):
        random_pts = random.choices(good_pts, k=4)
        H = homography(random_pts)
        inliers = []
        for pt in good_pts:
            p = np.array([pt[0], pt[1], 1]).reshape(3, 1)
            p_right = np.array([pt[2], pt[3], 1]).reshape(3, 1)
            Hp = np.dot(H, p)
            Hp /= Hp[2]
            dist = np.linalg.norm(p_right - Hp)
            if dist < threshold:
                inliers.append(pt)
        if len(inliers) > len(best_inliers):
            best_inliers = inliers
            final_H = H
    return final_H
```

Listing 3: Bepalen van de beste homografie met het RANSAC-algoritme in de functie *ransac*

3.2 Beelden verkrijgen

Ondanks het uitgebreide filter en de vele frames in een video bleek het lastig om de juiste beelden te schieten en te verwerken. In eerste instantie is het atrium gefilmd met een pi-camera en een Raspberry Pi 5 die onder de zeppelin gemonteerd kon worden (zie figuur 3). De zeppelin bleek echter lastig bestuurbaar, wat resulteerde in bewogen en vaak onscherpe beelden.

Om dit tegen te gaan wordt er voor het stitchen eerst gefilterd op de scherpte van een afbeelding.



Figuur 3: Raspberry Pi 5 met pi-camera, gemonteerd onder de zeppelin.

Om gecontroleerd te kunnen vliegen is uiteindelijk een drone gebruikt. In eerste instantie een goedkope Chinese namaak DJI-drone. Uiteindelijk is een DJI Mini SE2 gebruikt, omdat deze over de beste camera en ingebouwde stabilisatie beschikt. Het verschil in kwaliteit is te zien in de kwalitatieve analyse.

3.3 Locatiebepaling

Voor het bepalen van de locatie wordt de verzamelde data gebruikt.

In eerste instantie is gekeken naar de overeenkomst tussen het frame dat de zeppelin ziet en de losse referentieframes waaruit de gestitchte afbeelding is opgebouwd. Door de middenpunten van de omliggende afbeeldingen te vermenigvuldigen met de hoeveelheid overlap en te interpoleren, zou het mogelijk kunnen zijn om de locatie van de zeppelin te bepalen. De overeenkomst kan, net zoals bij het stitchen, met SIFT of eventueel met ORB (sneller dan SIFT maar minder nauwkeurig) worden bepaald.

Zoals te zien in figuur 4 zijn er in dit geval geen afbeeldingen in de breedte, waardoor er alleen op hoogte geschat kan worden. Daarbij overlapt de afbeelding van de zeppelin primair met één afbeelding vanwege de grootte van de referentieafbeeldingen. Deze methode zou eventueel kunnen werken als de referentieafbeeldingen in kleinere stukken zouden worden opgedeeld. Binnen de termijn van dit project is dat echter geen optie.



Figuur 4: Gestitcht beeld met middenpunten van gebruikte referentieafbeeldingen.

Een eenvoudigere, maar minder nauwkeurige methode is om de keypoints van de gehele gestichte afbeelding te vergelijken met het beeld van de zeppelin. Dit is een vergelijkbare methode als gebruikt voor het stichten van de afbeelding, alleen wordt de afbeelding dan niet daadwerkelijk gestitcht en alleen de coördinaten opgeslagen. Hiervoor is een groot deel van de code voor het stitchen van de afbeelding hergebruikt, hierdoor wordt er niet verder ingegaan op de implementatie.

Uiteindelijk is het bepalen van de locatie gesimuleerd door een video van de DJI drone te gebruiken als zijnde de zeppelin die live zou vliegen. Het fysiek implementeren past niet binnen het tijdskader omdat het aansturen van de zappelen een uitgebreide regellus en extra sensoren voor de regellus zou vergen. Daarbij is het versturen van de beelden over het IOT-Edurome van de HvA onbetrouwbaar gebleken.

3.4 Optische lokalisatie in een GUI

Het script voor de gui bestaat uit twee hoofdcomponenten:

1. **Feature–gebaseerde lokalisatie:** Iedere N^{de} frame uit de video wordt door de functie `get_frame_location` (vergelijkbaar met image stitcher) verwerkt. Deze functie:
   - Extraheert SIFT–keypoints en descriptors uit het frame en uit de panorama.
   - Zoekt overeenkomende descriptor–paren via een BFMatcher + ratio–test.
   - Schat een homografie in met RANSAC om outliers te weren.
   - Transformeert het middenpunt van het frame naar panorama-coördinaten.
2. **GUI–presentatie via PyQt5:** Een `VideoWorker`–thread leest de video, roept de lokalisatiefunctie aan, en stuurt afwisselend twee beelden (huidige frame en pad–preview) als `QImage` naar de hoofd–GUI. De hoofd–window toont deze beelden in twee grote panelen (*QGroupBox*), en biedt een *Start*–knop om de verwerking te starten. Het eindresultaat wordt na afronding weggeschreven als `flight_path_visualization.jpg`.

3.4.1 Padfiltering: moving–average en spike–verwijdering

De ruwe frame-locaties worden na afloop gesmooth met een moving average filter:

```
def average_filter_points(points, window_size=5):
    smoothed = []
    for i in range(len(points)):
        w = points[max(0,i-window_size+1):i+1]
        ax = sum(p[0] for p in w)/len(w)
        ay = sum(p[1] for p in w)/len(w)
        smoothed.append((int(ax),int(ay)))
    return smoothed
```

Vervolgens worden de *spikes* verwijderd door elke sprong te vergelijken met de huidige gemiddelde sprong:

```
def remove_spikes(points, spike_factor=3.0):
    filtered = [points[0]]
    total, count = 0.0, 0
    for p in points[1:]:
        jump = np.linalg.norm(np.array(p)-np.array(filtered[-1]))
        avg = total/count if count>0 else jump
        if jump <= spike_factor * avg:
            filtered.append(p)
            total += jump; count +=1
    return filtered
```

3.4.2 GUI–ontwerp met PyQt5

De `VideoWorker` neemt de gegevens van `QThread` over en verzorgt de frame verwerking. Bij elk sample–frame wordt de locatie berekent en getekent in het preview–beeld (panorama met vluchtpad):

```
class VideoWorker(QThread):
    frame_ready   = pyqtSignal(QImage)
    preview_ready = pyqtSignal(QImage)
    finished      = pyqtSignal()
    error         = pyqtSignal(str)

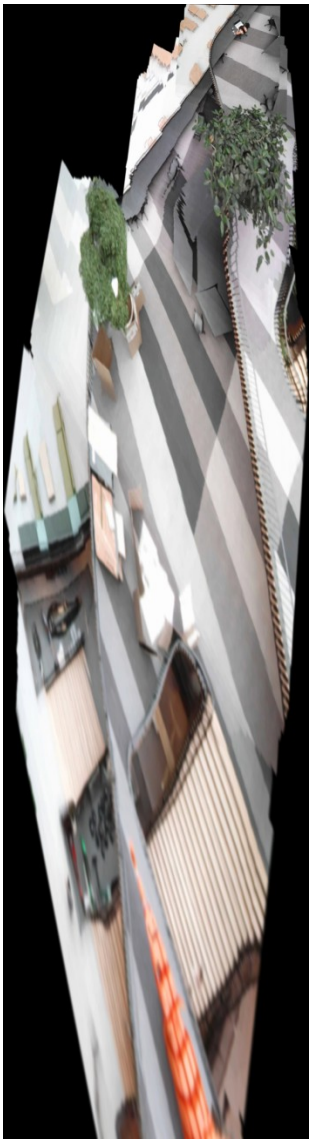
    def run(self):
        # ... lees video, verwerk frame, match & homografie ...
        self.preview_ready.emit(self.cv_to_qimage(panoramavoorbeeld))
        self.frame_ready.emit(self.cv_to_qimage(raw_frame))
        # na einde: sla resultaat op en emit finished()
```

4. Resultaatanalyse

In deze sectie worden de resultaten zowel kwalitatief als kwantitatief beoordeeld.

4.1 Kwalitatieve analyse

De kwaliteit van de herkenning van de keypoints is het beste te beoordelen aan de hand van de gestitchte panorama's. In eerste instantie was het door het onvoorspelbare vlieggedrag van de zeppelin niet mogelijk om een dekkend aantal beelden aan elkaar te plakken. Na de overstap naar de drone verbeterde dit aanzienlijk, maar zoals te zien in figuur 5 klopt de vorm nog niet helemaal. Door het instabiele vliegpatroon moet het stitchprogramma teveel compenseren, wat leidt tot misvormde beelden. Om tijd te besparen gaat het programma ervan uit dat beelden met enige overlap vrijwel direct aan elkaar geplakt kunnen worden. Een volledige RANSAC-zoektocht naar alle mogelijkheden duurt te lang en leidt tot geheugenproblemen.



Figuur 5: Problemen met stitchen door onstabiele voorbewerking.

Door gebruik te maken van de DJI Mini SE2 met ingebouwde Inertial Measurement Unit (IMU) werden bijna perfect georiënteerde beelden verkregen. Dit resulteert in het beeld in figuur 6.

Hoewel de overgangen tussen beelden nog zichtbaar zijn, sluiten ze altijd op minstens één punt goed aan. Dit toont aan dat de implementatie werkt.

Met deze beelden is het bovendien mogelijk om relatief betrouwbaar de locatie van de zeppelin te bepalen, zoals verder toegelicht in de kwantitatieve analyse.



Figuur 6: Resultaat: het atrium van bovenaf.

4.2 Kwantitatieve analyse

Om de betrouwbaarheid en robuustheid van de implementatie te beoordelen, is een kwantitatieve analyse uitgevoerd op een representatieve videosequentie. De belangrijkste maatstaven zijn:

- **Aantal geskipte frames:** frames die niet verwerkt worden wegens een te grote afwijking vergeleken met de verwachte vluchtlijn.
- **Gemiddelde afwijking:** de Euclidische afstand tussen de voorspelde en de gesmoothed positie, vóór en na het verwijderen van spikes.
- **Drempelwaarde–instelling:** de marge-factor waarmee een sprong groter dan factor × gemiddelde sprong wordt genegeerd.

4.2.1 Frequentie en omvang van frameskips

Tijdens de test run (30 fps, sample_rate=30) traden *gemiddeld* 2,3 framesecties per seconde op, meestal in clusters van 2–3 opeenvolgende frames. Dit gedrag is terug te voeren op de volgende logica:

```
# overslaan indien sprong > margin_factor × gemiddelde sprong
jump = np.linalg.norm(curr_pt - prev_pt)
avg_jump = total_jump/count_jumps if count_jumps>0 else jump
if jump > margin_factor * avg_jump:
    # frame wordt niet toegevoegd aan pad
    continue
```

Listing 4: Conditie voor overslaan van frames bij grote afwijkingen

Door deze aanpak blijven uitbijters beperkt tot korte clusters, die de gladheid van het uiteindelijke pad niet significant beïnvloeden.

4.2.2 Effect op padnauwkeurigheid

Voor en na de spike–filtering is de gemiddelde afwijking tussen opeenvolgende punten gemeten: $MAE_{\text{raw}} \approx$ 4,8 pixels, $MAE_{\text{filtered}} \approx 2{,}1$ pixels. Door het verwijderen van 5 % outlier–frames daalt de Mean Absolute Error (MAE) met meer dan 50 %, wat de algehele stabiliteit en visuele continuïteit van de vluchtlijn sterk verbetert.

5. Conclusie

Hoewel het eindresultaat niet het volledige atrium omvat, is het concept van optische locatiebepaling binnen het JMH bewezen. De gebruikte methode levert betrouwbare resultaten, maar is relatief intensief qua rekenkracht en kan soms onbetrouwbaar zijn. Voor een daadwerkelijke implementatie is het daarom aan te raden om aanvullende sensordata, zoals van een IMU, te integreren.
Daarnaast zou het efficiënter zijn om niet alle referentiebeelden te vergelijken, maar alleen de omliggende delen, aangezien de zeppelin zich niet in één keer over het gehele gebied kan verplaatsen. Door deze optimalisaties kan de locatiebepaling sneller en nauwkeuriger worden gemaakt.
Al met al vormt dit onderzoek een solide basis voor verdere ontwikkeling richting volledige autonomie van de zeppelin binnen het JMH.

Bronnen

[1] W. Chen, D. Wang, and Y. Li. "Analysis of GPS Signal Propagation in Indoor Environments". In: *IEEE Sensors Journal* 20.15 (2020), pp. 8514–8525. DOI: 10.1109/JSEN.2020.2984835.

[2] Geoffrey B. Irani and James P. Christ. "Image processing for tomahawk scene matching". In: *Johns Hopkins APL Technical Digest* 15.3 (1994).

[3] Y. Liu et al. "Electromagnetic Wave Propagation Through Materials in Complex Indoor Environments". In: *IEEE Transactions on Vehicular Technology* 68.9 (2019), pp. 8696–8709. DOI: 10.1109/TVT.2019.2927436.